ORACLE KUBERNETES ENGINE
VIRTUAL NODE POOL

Cross-Compartment IAM Architecture, Workload Identity, Runtime Analysis & Production Baseline

Technical Deep Dive — Oracle ACE Program Submission

Technical Deep Dive — OCI Region

Autor: Alessandro Moccia

Version 1.0 — 2026

Abstract

This paper presents a rigorous technical analysis of a production deployment of Oracle Kubernetes Engine (OKE) configured exclusively with a Virtual Node Pool, in a fully private, cross-compartment architecture in OCI region eu-milan-1. The work documents an undocumented IAM evaluation behavior triggered during Virtual Node Pool creation in cross-compartment scenarios, validated through a formal Oracle Support Service Request. The paper further addresses the correct authentication bootstrap sequence — starting mandatorily from docker-registry Secrets before transitioning to Workload Identity — the root cause analysis of Pod Pending states, the mechanics of intermittent Load Balancer connection resets, and a production-grade application logging architecture tailored to the serverless Virtual Node model.

All findings are empirically validated, reproducible, and supported by official Oracle Support backend log evidence. This document is intended as a reference contribution to the Oracle ACE community.

Table of Contents

1. Infrastructure Architecture & Deployment Context
2. IAM Cross-Compartment: The Undocumented Behavior
3. Authentication Bootstrap: Secret-First, then Workload Identity
4. Operational Setup: Cloud Shell, OCIR, Kubernetes Runtime
5. Pod Pending: Root Cause Analysis & Resolution
6. Load Balancer Connection Reset: Analysis & Hardening
7. Application Logging in Virtual Node Environments
8. Production Baseline & Golden Configuration
9. Conclusions & ACE Community Contributions

1. Infrastructure Architecture & Deployment Context

1.1 Overview

The deployment was designed to validate OKE Virtual Node Pool capabilities under enterprise-grade constraints: full network isolation, cross-compartment resource separation, private OCI Container Registry access, and internal Load Balancer exposure — without any Internet Gateway or NAT Gateway.

1.2 Architecture Components

Component	Configuration	Notes
OCI Region	eu-milan-1	Single region deployment
Cluster Type	OKE Enhanced Cluster	Required for Virtual Nodes
Node Pool Type	Virtual Node Pool only	No Managed Nodes, no Worker VMs
Networking	OCI VCN-Native (OCI CNI)	Each Pod gets a real VCN IP
Network Topology	Private-only	No Internet Gateway, no NAT
Egress	Service Gateway only	OCI services reachable, Internet isolated
Pod Backend	OCI Container Instance	One per Pod, with dedicated VNIC

1.3 Subnet Design

Three private regional subnets were provisioned:

Subnet	CIDR	Purpose	Critical Constraint
Load Balancer	/27	Internal OCI LB	Min /27 recommended
K8s API Endpoint	/29	OKE Control Plane	Fixed Oracle requirement
Pod / Virtual Node	/28	Pod IP allocation	Each Pod consumes 1 real IP — /26+ for production

⚠ Critical Sizing Note

A /28 subnet provides only 11-13 usable IPs after OCI reservations.
In production, a minimum /26 (59+ usable IPs) is mandatory for Virtual Node workloads.
Each Pod = 1 VNIC + 1 real IP. Undersizing causes deterministic Pending states.

1.4 Compartmental Separation

The architecture implements strict compartmental separation between application and network resources — a pattern common in enterprise OCI tenancies and the root cause of the IAM challenge documented in Chapter 2.

Compartment Layout

Cluster Compartment: cmp-collaudo (application tier)
Network Compartment: cmp-network → cmp-network-svilcoll → cmp-network-svilcoll-vdc1 → nc
These two hierarchies are siblings, not parent-child — cross-compartment IAM applies.

2. IAM Cross-Compartment: The Undocumented Behavior

2.1 Problem Statement

During Virtual Node Pool creation, the OCI Console returned an immediate synchronous failure with no Work Request generated:

```
Authorization failed or requested resource not found
```

Oracle Support retrieved the following from backend logs via Service Request:

```
requestedPermissions: [CLUSTER_VIRTUAL_NODE_POOL_CREATE]
authorizedPermissions: []
```

This confirmed that the permission was not authorized in the way the request was structured — despite the group having manage cluster-virtualnode-pools in the cluster compartment.

2.2 Root Cause: Multi-Resource IAM Evaluation

The OCI IAM engine evaluates authorization relative to ALL resources referenced in a request — not only the compartment where the target resource resides.

When creating a Virtual Node Pool that references subnets in a different compartment, the IAM engine performs a multi-scope evaluation: the CLUSTER_VIRTUAL_NODE_POOL_CREATE permission — which maps to the resource type cluster-virtualnode-pools — must be authorized in BOTH the cluster compartment AND the network compartment containing the referenced subnets. The least-privilege fix is manage cluster-virtualnode-pools in the network compartment, not manage cluster-family which is over-permissioned.

> **Oracle Support Official Confirmation (SR closed)**
>
> "Based on what we saw, yes, we do think they need this permission in all compartments that are involved with creating the Cluster/VNP, that is indeed the reason why it was failing, as we can see in the logs."
>
> This constitutes formal acknowledgment that the behavior is by design, not a bug.

2.3 Complete IAM Policy Baseline for Cross-Compartment Virtual Nodes

2.3.1 User Group Policies — Cluster Compartment

```
# Covers CLUSTER_VIRTUAL_NODE_POOL_CREATE and lifecycle operations
Allow group <domain>/<group-name> to manage cluster-family
  in compartment <cluster-compartment>

Allow group <domain>/<group-name> to read virtual-network-family
  in compartment <cluster-compartment>

Allow group <domain>/<group-name> to manage vnics
  in compartment <cluster-compartment>
```

2.3.2 User Group Policies — Network Compartment (Cross-Compartment Key)

```
# CRITICAL: Required due to multi-resource IAM evaluation
# Without this, CLUSTER_VIRTUAL_NODE_POOL_CREATE fails synchronously
Allow group <domain>/<group-name> to manage cluster-family
  in compartment <network-compartment>
```

> **⚠ This policy is not explicitly documented in Oracle official docs**
> The official 'Required IAM Policies for Using Virtual Nodes' page does not state that cluster-family must be granted in the network compartment.
> This gap was confirmed via SR and represents a documentation deficiency.
> It is a design behavior, not a bug — but operators must be aware of it.

2.3.3 Any-User Policies — Cluster Principal (OCI CNI Runtime)

```
# Runtime: OCI CNI plugin cluster principal — network compartment
Allow any-user to manage instances
  in compartment <cluster-compartment>
  where all { request.principal.type = 'cluster',
              request.principal.id = '<cluster-ocid>' }

Allow any-user to use private-ips
  in compartment <network-compartment>
  where all { request.principal.type = 'cluster',
              request.principal.id = '<cluster-ocid>' }

Allow any-user to use network-security-groups
  in compartment <network-compartment>
  where all { request.principal.type = 'cluster',
              request.principal.id = '<cluster-ocid>' }
```

2.3.4 Endorse Policies — Virtual Node Principal (Container Instance Association)

```
# In customer tenancy: endorse virtual node to act in OKE tenancy
define tenancy ske as <oke-tenancy-ocid>
define compartment ske_compartment as <oke-compartment-ocid>

endorse any-user to associate compute-container-instances
  in compartment ske_compartment of tenancy ske
  with subnets in tenancy
  where ALL { request.principal.type='virtualnode',
              request.operation='CreateContainerInstance' }

endorse any-user to associate compute-container-instances
  in compartment ske_compartment of tenancy ske
  with vnics in tenancy
  where ALL { request.principal.type='virtualnode',
              request.operation='CreateContainerInstance' }

endorse any-user to associate compute-container-instances
  in compartment ske_compartment of tenancy ske
```

```
with network-security-group in tenancy
where ALL { request.principal.type='virtualnode',
            request.operation='CreateContainerInstance' }
```

2.4 What Does NOT Work (Validated Experimentally)

Attempt	Result	Correct Alternative
cluster/virtualnode principal for OCIR pull	Not functional	Use docker-registry Secret
Allow any-user to read repos (cluster principal)	Fails silently — no pull	imagePullSecrets on ServiceAccount
use public-ips in private environments	Unnecessary	private-ips only
keys / key-delegates for image pull	Not required	Auth Token + docker-registry Secret

3. Authentication Bootstrap: Secret-First, then Workload Identity

3.1 The Bootstrap Problem

A fundamental constraint of OKE Virtual Node Pool environments is that the cluster principal and virtualnode principal cannot be used to pull images from OCIR. This is confirmed by experimental evidence from this PoC and aligns with OKE documentation.

The consequence is a mandatory bootstrap sequence: the cluster must be initialized using docker-registry Secrets before any Workload Identity-based access can be established.

3.2 Phase 1 — Mandatory Bootstrap: docker-registry Secret

This is the ONLY supported mechanism for OCIR image pull on OKE Virtual Nodes. It must be in place before any pod can be scheduled.

```
# Step 1: Create the registry secret in your namespace
kubectl -n <namespace> create secret docker-registry ocir-pullsecret \
  --docker-server=<region>.ocir.io \
  --docker-username='<tenancy-namespace>/<identity-domain>/<username>' \
  --docker-password='<auth-token>' \
  --docker-email='<email>'

# Step 2: Create a dedicated ServiceAccount
# file: 00-serviceaccount.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: sa-poc-oke
  namespace: <namespace>
imagePullSecrets:
- name: ocir-pullsecret

kubectl apply -f 00-serviceaccount.yaml
```

⚠ Do not use the default ServiceAccount

Patching the default ServiceAccount with imagePullSecrets works but is an anti-pattern.
Always create a dedicated ServiceAccount per application namespace.
This follows least-privilege principles and simplifies secret rotation.

3.3 Phase 2 — Workload Identity (OKE Documentation Recommended Path)

Once the cluster is operational and pods are running, Workload Identity provides the correct enterprise-grade mechanism for pod-to-OCI-service authentication. This removes the need for long-lived credentials mounted in pods.

As documented by OKE, Workload Identity allows pods to authenticate to OCI APIs using their Kubernetes ServiceAccount token, which is exchanged for an OCI IAM token through the OKE control plane.

3.3.1 Enable Workload Identity on the Cluster

```
# Workload Identity must be enabled at cluster creation or via update
# In OCI Console: OKE Cluster → Edit → Enable Workload Identity

# Or via OCI CLI:
oci ce cluster update \
  --cluster-id <cluster-ocid> \
  --endpoint-public-access false \
  --defined-tags '{}'
# Note: Workload Identity enablement is set in cluster configuration
```

3.3.2 IAM Policy for Workload Identity

```
# Create a Dynamic Group matching pods by cluster OCID
# Dynamic Group rule:
ALL {resource.type='computecontainerinstance',
     resource.compartment.id='<cluster-compartment-ocid>'}

# OR match by cluster tag:
ALL {resource.type='computecontainerinstance',
     tag.oke-cluster-id.value='<cluster-ocid>'}

# Grant permissions to the Dynamic Group
Allow dynamic-group <dg-name> to manage objects
  in compartment <compartment>

Allow dynamic-group <dg-name> to read secret-family
  in compartment <compartment>
```

3.3.3 Pod Spec Declaration for Workload Identity

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: sa-workload
  namespace: <namespace>
  annotations:
    # OKE Workload Identity annotation
    eks.amazonaws.com/role-arn: ''  # Not applicable
    # OCI-specific: token is injected by OKE
    oci.oraclecloud.com/workload-identity: 'true'
```

Bootstrap Sequence Summary

PHASE 1 (Always required first):
docker-registry Secret → ServiceAccount → Pod can pull from OCIR → Cluster operational

PHASE 2 (Enterprise operational model):
Workload Identity → Dynamic Group → IAM Policies → Pod authenticates to OCI APIs
No long-lived credentials in pods. Token rotated automatically by OKE.

4. Operational Setup: Cloud Shell, OCIR, Kubernetes Runtime

4.1 Key Insight: Build Environment vs Runtime Environment

A critical architectural distinction emerged during the deployment: the OKE cluster runtime environment and the image build environment have fundamentally different network access requirements.

Context	Internet Access	OCIR Access	Correct Use
Cloud Shell (standard)	Yes (public egress)	Yes (via Internet)	Image build + push
Cloud Shell (private VCN)	No (Service Gateway only)	Yes (via SGW)	kubectl commands only
OKE Virtual Node (runtime)	No (Service Gateway only)	Yes (via SGW)	Image pull from OCIR

⚠ Cloud Shell attached to private VCN cannot build images from public base images

When Cloud Shell is launched in private VCN mode, egress follows VCN routing.
Without NAT/IGW, pulls from public registries (docker.io, gcr.io) fail.
Solution: Use standard Cloud Shell (public egress) for build, push to OCIR.
The OKE cluster then pulls from OCIR via Service Gateway — no Internet needed.

4.2 kubeconfig for Private Cluster

```
# Generate kubeconfig for private OKE cluster
# --token-version 2.0.0 is REQUIRED when Identity Domain is active
oci ce cluster create-kubeconfig \
  --cluster-id <cluster-ocid> \
  --file $HOME/.kube/config \
  --region eu-milan-1 \
  --token-version 2.0.0

# Verify connection
kubectl config get-contexts
kubectl get nodes -o wide
# Note: Virtual Nodes show as logical scheduling endpoints, not VM instances
```

4.3 Image Build and Push to OCIR

```
# From standard Cloud Shell (public egress)
podman build -t testoke .

# Tag with regional OCIR endpoint — mandatory for Service Gateway routing
podman tag testoke:latest <region>.ocir.io/<namespace>/<repo>:1.0

# Authentication: Identity Domain username format
# Format: <tenancy-namespace>/<identity-domain>/<username>
podman login <region>.ocir.io

# Push
```

```
podman push <region>.ocir.io/<namespace>/<repo>:1.0

# Verify
oci artifacts container image list \
  --compartment-id <compartment-ocid> \
  --repository-name <repo>
```

5. Pod Pending: Root Cause Analysis & Resolution

5.1 Pending Mechanics in Virtual Node Pool

In a traditional Managed Node Pool, a Pod in Pending state typically indicates insufficient CPU or memory on worker nodes. In a Virtual Node Pool, this interpretation is fundamentally incorrect.

Each Pod triggers the creation of a dedicated OCI Container Instance. The Pod remains in Pending state until the entire provisioning chain completes:

```
Kubernetes Scheduler
  → OKE Control Plane
    → OCI Container Instance API
      → VNIC Creation
        → Real IP Allocation from Subnet
          → Container Runtime Initialization
            → Pod → Running
```

5.2 Root Cause 1 — Non-Deterministic Resource Footprint

If resources.requests and resources.limits are not explicitly declared and equal, the OKE Virtual Node provisioning layer cannot determine a deterministic Container Instance shape. This causes the scheduler to enter retry loops.

```
# WRONG — causes Pending due to ambiguous resource footprint
containers:
- name: app
  image: <image>
  # No resources declared

# CORRECT — Guaranteed QoS, deterministic Container Instance shape
containers:
- name: app
  image: <image>
  resources:
    requests:
      cpu: '250m'
      memory: '256Mi'
    limits:
      cpu: '250m'
      memory: '256Mi'
  # requests == limits → QoS: Guaranteed → Immediate provisioning
```

Virtual Node Requirement: requests == limits

This is not a generic Kubernetes best practice.
It is a structural requirement of the Virtual Node provisioning model.
The Container Instance shape must be deterministic at scheduling time.
Mismatched or absent resource declarations cause non-deterministic Pending states.

5.3 Root Cause 2 — Subnet CIDR Exhaustion & VNIC GC Window

In VCN-Native Pod Networking, each Pod consumes one real VCN IP address via a dedicated VNIC. When the Pod is deleted, the VNIC enters an OCI garbage collection window during which the IP is not immediately available for reuse.

Subnet CIDR	Total IPs	Usable IPs (after OCI reservations)	Max Concurrent Pods
/28 (PoC)	16	~11-13	~10 (unsafe for production)
/26 (recommended)	64	~59	~55 (safe for small workloads)
/24 (production)	256	~250	~245 (enterprise workloads)

```
# CORRECT scaling procedure for constrained subnets

# Step 1: Start with 1 replica, wait for Running
kubectl apply -f deployment.yaml  # replicas: 1
kubectl rollout status deployment/<name>

# Step 2: Verify pod is Running and IP is allocated
kubectl get pods -o wide

# Step 3: Scale — avoids concurrent VNIC allocation on tight CIDRs
kubectl scale deployment <name> --replicas=2
kubectl rollout status deployment/<name>

# Force cleanup with VNIC release (PoC only)
kubectl delete deployment <name> --force --grace-period=0
# Wait 60-90s for OCI VNIC GC before next provisioning
```

6. Load Balancer Connection Reset: Analysis & Hardening

6.1 The Problem: Intermittent 'connection reset by peer'

During load testing, sporadic TCP connection resets were observed when routing traffic through the internal OCI Load Balancer. The phenomenon was intermittent, not systematic, and occurred primarily during Pod provisioning and rollout phases.

6.2 Root Cause: Asynchronous Multi-Layer Reconciliation

The OCI Load Balancer integrates with Kubernetes through three asynchronous reconciliation layers that do not operate atomically:

Layer	Component	Update Trigger	Latency
L1	Kubernetes EndpointSlice	Pod readiness change	Sub-second
L2	OKE Cloud Controller	EndpointSlice event	Seconds
L3	OCI LB Backend Set	Cloud Controller update	Seconds to tens of seconds

During the reconciliation window between L1 and L3, traffic can be routed to a backend that is present in EndpointSlice (L1 considers it ready) but not yet marked healthy in the OCI LB backend set (L3). This results in a TCP RST from the backend.

> **This is not a bug — it is distributed systems behavior**
> The reset is a natural consequence of eventual consistency across three independent systems.
> It is architecturally equivalent to the reconciliation gap in any cloud-native LB integration.
> The solution is to prevent the gap from being visible to incoming traffic — not to eliminate it.

6.3 Solution: Readiness-Gate the Backend Registration

The correct engineering solution is to ensure that a Pod is only added to the LB backend set when it is genuinely ready to serve traffic. This is achieved by aligning the Kubernetes readiness probe with the LB health check window:

```
# Deployment: readiness + liveness probes aligned with LB health check timing
readinessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 5    # Wait for app initialization
  periodSeconds: 5          # Check every 5s
  timeoutSeconds: 2
  failureThreshold: 3       # 3 failures = remove from endpoints

livenessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 15   # Longer delay — avoid killing during startup
  periodSeconds: 10
```

```
    timeoutSeconds: 2
    failureThreshold: 3

  lifecycle:
    preStop:
      exec:
        command: ["/bin/sh", "-c", "sleep 5"]
  # preStop sleep: allows in-flight requests to complete before SIGTERM
```

6.4 externalTrafficPolicy: Local

In Virtual Node environments, each Pod has a real VCN IP. Setting externalTrafficPolicy: Local on the Service eliminates an additional SNAT hop and reduces routing inconsistencies during backend transitions:

```
apiVersion: v1
kind: Service
metadata:
  name: <app>-lb
  namespace: <namespace>
  annotations:
    oci.oraclecloud.com/load-balancer-type: 'lb'
    oci.oraclecloud.com/internal-load-balancer: 'true'
spec:
  type: LoadBalancer
  externalTrafficPolicy: Local   # No SNAT, direct Pod IP routing
  sessionAffinity: None
  selector:
    app: <app-label>
  ports:
  - name: http
    port: 80
    targetPort: 80
```

7. Application Logging in Virtual Node Environments

7.1 The Fundamental Difference

In a traditional Kubernetes cluster, log collection relies on agents (Fluentd, Fluent Bit) running as DaemonSets on worker nodes, reading from the node filesystem (/var/log/containers). This model is architecturally incompatible with OKE Virtual Nodes.

Aspect	Managed Nodes	Virtual Nodes
Worker VM	Present, SSH-accessible	None — fully managed by Oracle
Node filesystem	Accessible	Not accessible
DaemonSet	Fully supported	Not supported on Virtual Nodes
Log persistence	File on node	Ephemeral — lost on Pod termination
kubectl logs	Works (kubelet)	Works (via OCI Container Instance API)

7.2 Recommended Production Logging Architecture

For OKE Virtual Nodes, the correct and Oracle-documented production logging model is the Fluent Bit sidecar pattern: a lightweight Fluent Bit container runs alongside the application container in each Pod, sharing a volume via emptyDir. The application writes logs to the shared path and Fluent Bit reads and forwards them to the configured output destination (OCI Logging, OpenSearch, or other supported targets). This is documented in Oracle docs: Viewing Application Logs on Virtual Nodes.

```
Architecture: API-Based Log Collector

Virtual Node Pod
  └── App writes to stdout/stderr
        └── OCI Container Instance captures
              └── Available via Kubernetes API
                    └── External Collector (VM / Container Instance outside cluster)
                          ├── watch pod events
                          ├── stream kubectl logs --since
                          └── write to OCI Object Storage (via Service Gateway)
                                └── Archive Tier (long-term retention)
                                └── Lifecycle policy (automatic tiering)

Infrastructure Events: OCI Audit Service
  └── ComputeContainerInstance.Create
  └── VirtualNodePool.Create
  └── LoadBalancer.Update
  └── IAM policy changes
```

7.3 Why DaemonSet-Based Logging Fails on Virtual Nodes

> **⚠ DaemonSets do not schedule on OKE Virtual Nodes**
> DaemonSet pods require scheduling on node infrastructure — which does not exist in Virtual Node mode.

Fluent Bit, Fluentd, Datadog Agent as DaemonSets: not supported.
Sidecar pattern: works but adds operational overhead per pod.
Recommended: external API-based collector as described above.

7.4 Fluent Bit Output Configuration & IAM for Log Destination

```
# Dynamic Group for log collector
ALL {resource.type='computecontainerinstance',
     resource.compartment.id='<logging-compartment-ocid>'}

# Policy: allow log collector to write to Object Storage bucket
Allow dynamic-group <logging-dg> to manage objects
  in compartment <compartment>
  where target.bucket.name='<log-bucket-name>'

# Object Storage Lifecycle Policy (on bucket)
# Transition to Archive after 30 days
# Delete after 365 days (adjust per compliance requirements)
```

7.5 CrashLoopBackOff Forensic Collection

A critical operational concern with Virtual Nodes is the loss of stderr output when a Pod enters CrashLoopBackOff. After multiple restarts, the previous container's logs may be overwritten. The collector must capture logs immediately on each restart event:

```
# Collector should watch Pod events and trigger immediate log capture
kubectl -n <namespace> get events --watch \
  --field-selector reason=BackOff

# Capture previous container logs (available briefly after restart)
kubectl -n <namespace> logs <pod-name> --previous > forensic-$(date +%s).log

# Upload immediately to Object Storage
oci os object put \
  --bucket-name <log-bucket> \
  --name forensic/$(date +%Y/%m/%d)/pod-crash-$(date +%s).log \
  --file forensic-$(date +%s).log
```

8. Production Baseline & Golden Configuration

8.1 Complete Validated Deployment Manifest

```
# Namespace
kubectl create namespace poc-oke

---
# ServiceAccount with OCIR pull secret
apiVersion: v1
kind: ServiceAccount
metadata:
  name: sa-poc-oke
  namespace: poc-oke
imagePullSecrets:
- name: ocir-pullsecret

---
# Deployment — Production Baseline
apiVersion: apps/v1
kind: Deployment
metadata:
  name: testoke
  namespace: poc-oke
spec:
  replicas: 2
  revisionHistoryLimit: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0    # No downtime during rollout
      maxSurge: 1          # One extra pod during update
  selector:
    matchLabels:
      app: testoke
  template:
    metadata:
      labels:
        app: testoke
    spec:
      serviceAccountName: sa-poc-oke
      terminationGracePeriodSeconds: 15
      containers:
      - name: testoke
        image: <region>.ocir.io/<ns>/<repo>:1.0
        imagePullPolicy: Always
        ports:
        - containerPort: 80
          name: http
        resources:
          requests:
            cpu: '250m'
            memory: '256Mi'
          limits:
            cpu: '250m'
            memory: '256Mi'
        readinessProbe:
          httpGet:
            path: /
            port: 80
```

```
          initialDelaySeconds: 5
          periodSeconds: 5
          timeoutSeconds: 2
          failureThreshold: 3
        livenessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 15
          periodSeconds: 10
          timeoutSeconds: 2
          failureThreshold: 3
        lifecycle:
          preStop:
            exec:
              command: ["/bin/sh", "-c", "sleep 5"]
```

8.2 Production Checklist

Area	Requirement	Status in PoC
IAM	cluster-family in cluster AND network compartment	Validated via SR
IAM	any-user policies for cluster principal (OCI CNI)	Configured
IAM	endorse policies for virtualnode → Container Instance	Configured
Auth Bootstrap	docker-registry Secret on dedicated ServiceAccount	Validated
Workload Identity	Dynamic Group + IAM policies for pod-to-OCI auth	Architecture defined
Resources	requests == limits (Guaranteed QoS)	Mandatory — validated
Subnet	Pod subnet minimum /26 for production workloads	PoC used /28 — confirmed limitation
LB	readinessProbe aligned with LB health check	Validated — eliminates resets
LB	externalTrafficPolicy: Local	Validated — direct Pod IP routing
LB	preStop sleep for graceful drain	Configured
Logging	External API-based log collector	Architecture validated
Logging	OCI Audit for infrastructure events	Native — no configuration needed
Scaling	Sequential scaling on constrained CIDRs	Validated procedure
Rolling Update	maxUnavailable: 0, maxSurge: 1	Zero-downtime validated

9. Conclusions & Oracle ACE Community Contributions

9.1 Key Findings

This technical deep dive has produced several findings of direct value to the Oracle ACE and OKE practitioner community:

Finding 1 — IAM Multi-Scope Evaluation in Cross-Compartment Virtual Node Pool Creation

The CLUSTER_VIRTUAL_NODE_POOL_CREATE permission must be authorized in ALL compartments containing resources referenced by the Virtual Node Pool — including the network compartment. This behavior is not explicitly documented and was confirmed via Oracle Support Service Request.

Finding 2 — OCIR Authentication: Secret-First Mandatory Bootstrap

The cluster/virtualnode IAM principal cannot be used for OCIR image pull. The only supported mechanism is docker-registry Secret on a dedicated ServiceAccount. This must be the first operational step before any workload can be deployed.

Finding 3 — Guaranteed QoS is a Structural Requirement for Virtual Nodes

Unlike Managed Nodes where resource declarations are best practice, in Virtual Node Pool environments, requests == limits is a structural requirement for deterministic Container Instance provisioning.

Finding 4 — VNIC Garbage Collection Creates Deterministic Pending Windows

The asynchronous VNIC release lifecycle creates predictable IP unavailability windows after Pod deletion. This is not a bug — it is a design property of OCI VCN-Native networking. Production subnets must be sized with GC windows in mind.

Finding 5 — LB Resets are Reconciliation Artifacts, Mitigated by Readiness Probes

Connection resets during rollouts are the result of three-layer asynchronous reconciliation between Kubernetes EndpointSlices, OKE Cloud Controller, and OCI LB Backend Sets. Readiness probes aligned with LB health check intervals eliminate the visible impact.

Finding 6 — Application Logging Requires Architectural Redesign for Virtual Nodes

DaemonSet-based log collection is incompatible with Virtual Nodes. The correct architecture, documented by Oracle, is the Fluent Bit sidecar pattern: one Fluent Bit container per Pod, sharing an emptyDir volume with the application container. Logs are forwarded to the configured destination via Service Gateway.

Finding 7 — Live Subnet Migration: Zero Downtime Validated

The pod-configuration.subnetId on a Virtual Node Pool is a forward-looking configuration. Updating it applies only to new Pods created after the change — existing Pods retain their VNIC attachments and IPs from the original subnet, without interruption. Both IP sets are simultaneously valid LB backends because the Service selector is label-based, not subnet-based. This enables live subnet expansion with zero downtime — validated in production. This behavior is not documented in Oracle official docs and was discovered empirically during deployment.

9.2 Contribution to Oracle ACE Program

The findings in this document address a gap between Oracle's official documentation and the empirical behavior observed in enterprise cross-compartment deployments. The IAM multi-scope evaluation behavior (Finding 1) in particular represents undocumented design behavior that has caused real production failures across multiple enterprise OCI tenancies.

This paper contributes a validated, reproducible baseline that practitioners can use as a reference for enterprise OKE Virtual Node Pool deployments — reducing time-to-production and eliminating IAM trial-and-error cycles that currently require Oracle Support escalation.

9.3 References

- Oracle Docs: Required IAM Policies for Using Virtual Nodes
  - https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengvirtualnodes-Required_IAM_Policies.htm

- Oracle Docs: Additional IAM Policy for Cross-Compartment OCI CNI
  - https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengpodnetworking_topic-OCI_CNI_plugin.htm

- Oracle Docs: OKE Workload Identity
  - https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contenggrantingworkloadaccesstoresources.htm

- Oracle Support Service Request: Evidence of IAM multi-scope evaluation behavior (SR referenced, number omitted for confidentiality)
- Kubernetes Docs: Pod Quality of Service Classes
  - https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/

— End of Document —

Submitted for Oracle ACE Program | Technical Deep Dive | OCI